

WashMate Campus

编程大作业说明文档

武子皓、王成思宇、林恺鋆、王博翔、周逸航

2026-05-31

课程名称：大模型与生成式人工智能

作业名称：编程大作业

仓库地址：<https://github.com/vegetable123q/Wash-agent>

APK 路径：`frontend/android/app/build/outputs/apk/debug/app-debug.apk`

目录

1	项目概述	4
1.1	文档用途	4
1.2	问题背景与目标用户	4
1.3	已实现的产品范围	4
2	需求与评分点对应关系	5
3	系统功能展示	6
3.1	页面截图预留	6
3.2	Demo 演示流程	6
4	技术架构与模块设计	6
4.1	技术栈	6
4.2	核心目录	7
4.3	运行时形态	8
4.4	架构流程图	8
4.5	模块边界原则	9
5	核心实现说明	9
5.1	应用入口与导航	9
5.2	页面职责	9
5.3	前端 API/service 模块	10
5.4	本地状态与外部访问边界	11
5.5	后端参考实现	11
5.6	数据结构与主流程	13
6	外部服务、安全与配置	14
6.1	外部服务边界	14
6.2	在线数据来源与字段	14
6.3	密钥与本地配置	15
6.4	不静默猜测业务数据	15
7	运行方法与交付文件	15
7.1	本地运行与构建命令	15
7.2	自动检查与发布	16
8	测试与验收结果	16
8.1	自动化测试与覆盖	16
8.2	实测验收结果	16

目录	3
9 团队分工	17
9.1 成员与贡献预留	17
9.2 模块维护说明	17
9.3 开发注意事项	18
10 当前限制与未来改进	18
10.1 总结	18

1 项目概述

1.1 文档用途

本文档介绍 WashMate Campus 的产品目标、主要功能、前端实现、后端参考实现、数据结构、外部服务、测试验证和发布流程。它可以用于了解产品全貌，也可以作为后续维护和功能交接的说明材料。

WashMate Campus 不是“网页前端 + 长期运行的 Python HTTP 后端”的传统结构。当前稳定的用户侧产品是 `frontend/` 下的 `React + TypeScript + Vite + Capacitor` 移动端应用。核心业务服务以内置 TypeScript 模块的形式运行在 APK 内部，主要位于 `frontend/src/api/`。`backend/` 下的 Python 模块是参考实现和测试基准，用于明确模块边界、业务规则、数据结构和集成测试行为。

1.2 问题背景与目标用户

校园公共洗衣场景中，学生经常需要同时判断多个问题：今天是否需要洗衣，哪些衣物可以一起洗，哪些衣物不能烘干，宿舍楼下还有没有空闲机器，排队要等多久，本次洗衣大概要花多少钱，洗完后该如何晾晒或取衣。以上判断涉及衣物材质、颜色、洗护标签、穿着次数、个人偏好、机器实时状态、天气与晾晒条件等多类信息。

WashMate Campus 的目标是把这些信息放进同一个手机流程。用户维护自己的衣柜和宿舍楼配置后，应用能够给出可执行的洗衣建议：什么时候洗、洗哪些、怎么分桶、用什么机器程序、是否需要洗衣袋、是否适合烘干、预计费用和时间是多少，以及有哪些风险提醒。

1.3 已实现的产品范围

当前移动端已经覆盖以下页面和流程：

- 今日建议 (Today)：展示天气、脏衣篮状态、频率建议、方案入口和刷新状态。
- 穿搭 Wiki (Outfit Wiki)：基于衣柜、天气和穿搭记录生成搭配建议，并支持加入脏衣篮。
- 记录穿搭 (Record Outfit)：记录当天穿过的衣物，并自动增加相关衣物的穿着次数。
- 衣柜 (Wardrobe)：展示本地衣柜，支持查看详情、删除和清空。
- 添加衣物 (Add Clothing)：支持手动录入、本地图片、ModelHub 图片识别和文字识别。
- 衣物详情 (Clothing Detail)：展示材质、颜色、洗护记忆、风险、穿着次数和加入脏衣篮操作。
- 脏衣篮 (Dirty Basket)：选择本次要洗的衣物，形成洗衣规划模块的输入。
- 洗衣房 (Laundry Room)：展示宿舍楼机器状态、排队估计、价格模式和天气上下文。
- 机器详情 (Machine Detail)：展示选中机器的用户可理解信息。
- 方案详情 (Plan Detail)：展示分桶、程序、费用、时间和风险提醒，并支持确认执行、完成提示和撤销。

- 报告 (Report): 展示结构化洗衣报告、现在照做的简短行动说明、本周已完成洗衣统计、风险说明和节水节电说明。
- 我的 (Profile): 保存宿舍楼、楼层、取衣时间、烘干偏好; ModelHub 配置和演示数据入口默认收在高级设置中。

最新移动端脚本还补强了几处执行体验: Add Clothing 在识图按钮旁直接提示缺少 ModelHub 配置; Outfit Wiki 在缺少裤装或裙装时给出添加入口; Laundry Room 会展示设备编号, 并高亮当前方案推荐使用的机器。完成一次洗衣后, 应用会在本地保存完成记录, 让报告页在脏衣篮清空后仍能显示本周统计和最近一次洗衣内容。

2 需求与评分点对应关系

本项目按照 `evaluation_system.md` 中的最终交付评测体系整理。评分点和文档、代码、测试的对应关系如下。

评分模块	分值	本项目对应内容	主要文件或验证方式
后端核心功能	25	衣物抽取、衣柜、校园上下文、洗衣规划和报告生成完整串联。	<code>backend/</code> 、 <code>tests/</code> 、完整集成测试。
数据可信度	15	材质、天气、机器、价格和时长均来自明确来源; 缺失时显示错误或未知状态。	<code>config/machine_rules.json</code> 、 <code>campusMachineApi.ts</code> 、 <code>weatherService.ts</code> 。
移动端流程	20	Today、Wardrobe、Add Clothing、Laundry Room、Plan Detail、Report、Profile 等主流程可用。	<code>frontend/src/screens/</code> 、前端集成测试、APK 冒烟测试。
UI 与交互质量	15	页面有空态、加载态、错误态; 按钮有明确行为或禁用原因。	Vitest 页面测试、人工验收流程。
边界与异常	15	API 失败、配置缺失、空衣柜、机器不可用、天气不可用等状态显式展示。	<code>evaluation_system.md</code> 边界清单、前后端异常测试。
工程质量	10	测试、构建、APK、文档、CI 和模块边界完整。	<code>README.md</code> 、 <code>docs/structure_design.md</code> 、GitHub workflows。

3 系统功能展示

3.1 页面截图预留

以下位置用于放置最终页面截图。图片文件可在本地补充后替换占位框。建议截图覆盖 Today、Wardrobe、Add Clothing、Laundry Room、Plan Detail 和 Report。



3.2 Demo 演示流程

- docs/e_demo_script.md 中的 3-5 分钟演示流程可以浓缩为以下路径：
1. 打开移动端 App，查看 Today 页的天气、脏衣篮、洗衣建议和方案入口。
 2. 进入 Wardrobe 或 Add Clothing，查看样例衣物或添加新衣物。
 3. 在 Dirty Basket 中选择本次要洗的衣物。
 4. 进入 Laundry Room，查看宿舍楼机器状态、设备编号和排队估计。
 5. 打开 Plan Detail，查看分桶、程序、费用、时长和风险提醒。
 6. 打开 Report，查看行动步骤、本周完成记录、风险说明和节水节电说明。

4 技术架构与模块设计

4.1 技术栈

层级	技术	在本项目中的职责
Mobile UI	React 18, TypeScript, Vite	手机端页面、表单、状态、导航和用户交互。

APK shell	Capacitor 7, Android Gradle	Android WebView 容器、应用标识、Capacitor sync、debug/release APK 构建。
APK 内业务模块	<code>frontend/src/api/</code> TypeScript modules	衣柜、脏衣篮、完成洗衣记录、校园上下文、洗衣规划、报告生成、频率建议、识图请求和本地存储。
Python 参考实现	Python 3.12, dataclasses, unittest	参考业务实现、共享模型、规则验证和测试基准。
External services	ModelHub/Gemini v1beta, CleverSchool, Haier, Open-Meteo	衣物识别、宿舍机器状态和天气信息。
Tooling	uv, npm, GitHub Actions	依赖同步、测试、前端构建、Capacitor 同步、Android 构建和发布。

4.2 核心目录

```

Wash-agent/
  README.md
  AGENTS.md
  pyproject.toml
  app.py
  main.py
  backend/
    shared/models.py
    clothing_extraction/
    wardrobe/
    campus/
    laundry/
    reports/
  frontend/
    src/
      App.tsx
      api/
      components/
      screens/
      data/
    android/
    package.json
    capacitor.config.ts
  config/
    api_config.example.json
    machine_rules.json
  data/

```

```
wardrobe_sample.json
machines_mock.json
pics/
docs/
  structure_design.md
  c_delivery.md
  e_demo_script.md
  superpowers/plans/
tests/
  test_*.py
.github/workflows/
  preview.yml
  release-apk.yml
```

4.3 运行时形态

项目的实际运行时以移动端为主：

1. 用户打开 Capacitor 打包的移动端应用。
2. React 页面通过顶层 `frontend/src/App.tsx` 进行导航、状态编排和事件处理。
3. 页面调用 `frontend/src/api/` 中的 TypeScript service，读取本地衣柜、脏衣篮、用户配置和 ModelHub 配置。
4. 在用户配置宿舍楼后，校园上下文模块读取真实校园机器状态；天气模块读取 Open-Meteo。
5. 洗衣规划模块根据选中衣物、用户约束、校园上下文和价格规则生成洗衣方案。
6. report generator 把方案、完成洗衣记录和上下文转换为页面可直接渲染的报告。
7. 用户确认执行方案后，移动端先记录本次洗衣快照，再清空脏衣篮；用户可以在成功提示中撤销。

Python 后端不需要作为一个常驻 HTTP 服务被移动端调用。它的价值在于提供更清晰的模块化参考实现、共享业务语言和测试基准。

4.4 架构流程图

```
User
-> React / Capacitor screens
-> frontend/src/api/mobileSummary.ts
  -> userProfile and ModelHub config
  -> local wardrobe and dirty basket
  -> campusMachineApi + weatherService
  -> frequencyAdvisor
  -> laundryPlanner
```

```
-> reportGenerator
-> completed laundry records
-> mobile screens render plan, report, machines, and wardrobe state
```

Python backend

```
-> backend/shared/models.py defines shared data models
-> backend/* modules define reference behavior and tests
-> tests/ verifies extraction, wardrobe, campus, planning, report, and integration
```

4.5 模块边界原则

页面层只做交互和编排，不写 ModelHub 提示词、机器解析、JSON 存储细节、洗衣分桶规则和最终报告生成。跨模块数据通过明确的数据结构传递：Python 侧使用 `backend/shared/models.py`，移动端使用 `frontend/src/api/types.ts`。当已有模型能够描述数据时，不应新增平行结构或兼容旧路径。

5 核心实现说明

5.1 应用入口与导航

`frontend/src/App.tsx` 是移动端的应用编排中心，负责：

- 当前 screen id、浏览器历史和返回逻辑。
- 当前用户 profile、ModelHub config 和 mobile summary。
- 选中衣物 id、选中机器 id、最近完成的洗衣记录和刷新状态。
- 顶层事件，例如保存 profile、保存或清除 ModelHub 配置、删除衣物、选择脏衣篮、完成或撤销洗衣方案、保存穿搭记录等。

应用没有引入额外路由库，而是通过 screen id 和 parent tab 映射管理页面关系。例如 `planDetail` 和 `dirtyBasket` 属于 Today，`addClothing` 和 `clothingDetail` 属于 Wardrobe，`machineDetail` 属于 Laundry Room，`recordOutfit` 属于 Outfit Wiki。

5.2 页面职责

页面文件	职责
<code>TodayScreen.tsx</code>	展示当前摘要、天气、脏衣篮、频率建议和方案入口。
<code>DirtyBasketScreen.tsx</code>	选择或清空本次洗衣衣物，形成当前 laundry selection。
<code>PlanDetailScreen.tsx</code>	展示详细洗衣方案，使用对话框确认执行，并在完成后显示成功面板和撤销入口。

<code>OutfitWikiScreen.tsx</code>	展示穿搭分类、今日穿搭建议和加入脏衣篮操作；缺少下衣时直接提示添加裤装或裙装。
<code>RecordOutfitScreen.tsx</code>	记录当天穿搭，并把已穿衣物计入 wear count。
<code>WardrobeScreen.tsx</code>	展示衣柜列表，支持详情、删除和清空。
<code>AddClothingScreen.tsx</code>	手动新增衣物，或通过 ModelHub 图片/文字识别辅助填写；识图未配置时在按钮旁给出配置入口。
<code>ClothingDetailScreen.tsx</code>	展示单件衣物的材质、颜色、风险、洗护记忆和穿着次数。
<code>LaundryRoomScreen.tsx</code>	展示宿舍楼机器状态、设备编号、排队估计、天气、刷新状态和错误状态，并高亮方案推荐机器。
<code>MachineDetailScreen.tsx</code>	展示单台机器的状态、位置、价格和程序。
<code>ReportScreen.tsx</code>	渲染结构化报告、现在照做摘要、本周完成统计、风险提示和节约说明。
<code>ProfileScreen.tsx</code>	配置用户宿舍、楼层、取衣时间和烘干偏好；ModelHub 与演示数据放在高级设置中。

5.3 前端 API/service 模块

模块	职责
<code>mobileSummary.ts</code>	组合移动端页面需要的完整摘要，包括衣柜、脏衣篮、完成洗衣记录、天气、校园上下文、频率建议、洗衣方案、烘干方案和报告；同时负责完成方案和撤销完成记录。
<code>types.ts</code>	定义移动端共享类型，语义上镜像 Python <code>backend/shared/models.py</code> ，并包含 <code>CompletedLaundryRecord</code> 与 <code>CompletedLaundrySummary</code> 。
<code>userProfile.ts</code>	读取和保存宿舍楼、楼层、最晚取衣时间、烘干偏好等用户配置。
<code>modelHubConfig.ts</code>	保存、清除、校验 ModelHub 配置，并限制模型名。目前允许 <code>gemini-3.1-pro-preview</code> 。
<code>modelHubRecognition.ts</code>	在用户主动触发时发送图片或文字识别请求，要求结构化 JSON 输出。
<code>clothingExtractor.ts</code>	从手动输入或识别结果构造衣物信息，并转换为洗衣规划模块可使用的衣柜条目。
<code>campusTowerDirectory.ts</code>	提供页面可见的宿舍楼选项，隐藏 provider id。
<code>campusMachineApi.ts</code>	根据用户选择的宿舍楼构建校园机器上下文，归一化机器状态和排队估计。

<code>weatherService.ts</code>	读取清华附近 Open-Meteo 天气。
<code>pricingRules.ts</code>	提供洗衣、烘干、洗鞋程序的价格、时长和晾晒上下文规则。
<code>frequencyAdvisor.ts</code>	基于穿着次数、衣物类型和用户约束生成洗护频率建议。
<code>laundryPlanner.ts</code>	拆分洗衣桶、选择程序、分配机器、估算费用时间并生成提醒。
<code>reportGenerator.ts</code>	把 plan、drying plan、校园上下文和衣物信息转换为结构化报告。
<code>outfitLogStore.ts</code> <code>outfitWiki.ts</code>	/ 保存穿搭记录、统计搭配关系，并支持今日搭配建议。
<code>photoThumbnail.ts</code> <code>photoFileStorage.ts</code>	/ 生成并保存本地衣柜缩略图。

5.4 本地状态与外部访问边界

移动端把用户 profile、ModelHub 配置、衣柜、脏衣篮、完成洗衣记录、穿搭记录和本地图片引用保存在本机。普通衣柜、脏衣篮、计划、完成记录和报告流程不依赖独立后端服务。

完成洗衣记录保存在 `washmate.completedLaundryRecords`。每条记录包含完成时间、衣物 id、衣物名称、费用和时间估计、机器标签、方案摘要，以及完成前的穿着次数和洗涤次数快照。撤销时，移动端用该快照恢复计数，并把对应衣物放回脏衣篮。

只有三类行为会访问外部服务：

- 用户主动点击图片识别、批量识别或文字识别，并且已经配置 ModelHub。
- 用户保存宿舍楼后，应用读取 CleverSchool 或 Haier 洗衣机状态。
- 应用刷新摘要时读取 Open-Meteo 天气。

缺少配置、机器状态、天气、识别结果或价格规则时，系统应该展示明确的未配置、不可用、未知或错误状态，而不是静默补一个不存在的默认值。

5.5 后端参考实现

共享模型

`backend/shared/models.py` 是 Python 后端唯一的共享模型层，存放 enum 和 dataclass。它定义了从衣物输入、衣物 profile、衣柜 item、洗衣约束、校园上下文、洗衣方案、烘干方案到报告的完整数据语言。

契约分组	主要模型
洗护和风险	<code>RiskLevel</code> , <code>WashMethod</code> , <code>DryMethod</code> .

机器上下文	MachineType, MachineStatus, MachineTower, MachineInfo, MachineQueueEstimate, CampusContext.
衣物和衣柜	ClothingInput, LLMResponse, ClothingProfile, WashRecord, WardrobeItem.
规划	LaundryConstraints, FrequencyAdvice, LaundryChargeLine, LaundryBucket, LaundryPlan, DryingStep, DryingPlan.
报告	WashReport.

衣物信息抽取

`backend/clothing_extraction/` 负责把用户输入、商品信息、吊牌/OCR 文本和可选图片识别结果转换为结构化 `ClothingProfile`。

- `llm_client.py` 负责 ModelHub/Gemini v1beta 请求、图片内容、提示词和 JSON 结果格式。
- `product_info.py` 负责商品名、店铺名、吊牌文字、OCR、用户描述和补充来源归一化。
- `extractor.py` 负责解析模型输出、同步洗护动作、归一化材质比例、颜色、洗护标签、洗护符号、风险和缺失字段。

图片识别只发起一次带 JSON 格式约束的请求，使用 `responseMimeType=application/json` 和 `responseSchema`。结果包含图片类型、可见证据、材质、颜色、洗护动作、风险和缺失字段。该模块不负责衣柜存储、洗衣频率、校园机器、最终计划或报告。

衣柜记忆与洗护频率

`backend/wardrobe/store.py` 提供 `WardrobeStore`，负责衣柜 CRUD、洗涤历史、穿着次数、历史问题、输入校验和 JSON 序列化。`backend/wardrobe/frequency_advisor.py` 根据衣物、穿着次数和 `LaundryConstraints` 输出 `FrequencyAdvice`。

该模块的边界很清晰：它不调用模型，不读机器状态，也不生成最终洗衣分桶方案。

校园上下文

`backend/campus/machine_api.py` 提供 `LaundryMachineClient`，负责 CleverSchool 和 Haier 机器来源的请求与解析。它归一化宿舍楼、provider keys、机器 id、机器类型、状态、楼层、剩余时间和 provider 信息。

`backend/campus/context.py` 负责构建 `CampusContext`，把机器列表、天气、晾晒上下文和价格规则组合起来，并按机器类型生成 `MachineQueueEstimate`。如果用户选择的宿舍楼无法解析，或者调用方直接传外部楼栋 id 但没有说明 provider，该层应显式报错。

`backend/campus/weather.py` 负责读取并校验 Open-Meteo 天气数据。

洗衣规划

`backend/laundry/planner.py` 是规则化洗衣规划模块的参考实现。它负责：

- 校验本次选择的衣物和 urgent item。
- 按不可水洗、干洗、手洗、床品、深色或掉色风险、浅色标准桶、低风险混色桶等规则拆分。
- 根据 `CampusContext.available_machines` 分配洗衣机、烘干机或洗鞋机。
- 使用 `CampusContext.pricing_rules` 中的真实价格和时长计算 cost breakdown。
- 生成洗衣液用量、洗衣袋建议、防串色、防缩水、卫生和等待时间 warning。
- 在用户设置预算或最大等待时间时，在 `global_warnings` 中明确说明超预算、等待未知或等待超时。

`backend/laundry/load_estimator.py` 提供衣物负载估算和按负载拆分能力。`recommend_drying` 把烘干分配从洗涤分桶中拆开，使 wash plan 和 drying plan 的责任更清晰。

报告生成

`backend/reports/generator.py` 把 `LaundryPlan`、可选 `DryingPlan`、衣物列表和 `CampusContext` 转换为 `WashReport`。报告模块只解释洗衣规划模块已经生成的结果，不重新分桶、不重新决策、不抓取机器、不调用模型。

`WashReport` 保留 `sections`、`action_steps`、`cost_breakdown`、`savings_notes` 和 `risk_notes` 等结构化字段，前端可以直接渲染，不需要从自然语言报告中反向解析费用和步骤。

5.6 数据结构与主流程

数据结构对齐

Python 和 TypeScript 分别有自己的共享契约层：

- Python: `backend/shared/models.py`
- TypeScript: `frontend/src/api/types.ts`

TypeScript 数据结构保证 APK 内置业务服务可以独立运行。Python 数据结构保证后端参考实现和单元测试有稳定的数据语言。两者不要求文件内容逐字相同，但应保持业务语义一致。

核心业务数据流

1. 用户在 Profile 保存宿舍楼、楼层、取衣时间、烘干偏好和 ModelHub 配置。
2. 用户在 Add Clothing 手动录入衣物，或通过 ModelHub 识别辅助填写。
3. 衣物保存为本地 wardrobe item，包含材质、颜色、风险、洗护记忆和本地图片引用。

- 4. 用户在 Dirty Basket 中选择本次要洗的 item ids。
- 5. mobileSummary.ts 读取衣柜、脏衣篮、天气、机器和价格规则，生成统一 mobile summary。
- 6. frequencyAdvisor.ts 根据穿着次数和约束生成频率建议。
- 7. laundryPlanner.ts 根据选择衣物、校园上下文和价格规则输出 LaundryPlan。
- 8. 用户在 Plan Detail 确认执行后，mobileSummary.ts 写入完成洗衣记录，重置已洗衣物的穿着次数，增加洗涤次数，并清空本次脏衣篮。
- 9. Report 页面直接渲染当前方案报告；如果本次脏衣篮已经清空，则使用完成洗衣记录展示本周统计和最近一次洗衣。

6 外部服务、安全与配置

6.1 外部服务边界

服务	触发条件	用途
ModelHub / Gemini v1beta	用户配置凭据后主动点击图片识别、批量识别或文字识别。	把图片或文字证据转换为结构化衣物字段。
CleverSchool	用户保存支持的宿舍楼。	读取智慧校园洗衣机楼栋和机器状态。
Haier	用户保存支持的宿舍楼。	读取海乐生活点位和设备状态。
Open-Meteo	刷新 mobile summary。	读取清华附近天气，用于晾晒上下文和页面展示。

6.2 在线数据来源与字段

天气数据由移动端 frontend/src/api/weatherService.ts 和 Python 参考实现 backend/campus/weather.py 读取。当前接口是 Open-Meteo forecast API:<https://api.open-meteo.com/v1/forecast>。移动端使用清华附近坐标 40.0023, 116.3268, 时区为 Asia/Shanghai, 请求当前天气字段 temperature_2m、relative_humidity_2m、precipitation 和 weather_code。返回结果必须包含 current。温度、湿度和降水必须是有效数字。读取成功时，页面收到 source=open-meteo 和 status=live；读取失败或字段不合法时，页面收到 status=unavailable 和错误信息。

洗衣机数据来自两类校园服务。移动端先使用 frontend/src/api/campusTowerDirectory.ts 中的本地宿舍楼目录，把用户看到的宿舍楼名映射到内部 towerKey 或 positionId。页面不会直接展示这些内部 id。

CleverSchool 机器状态由移动端 frontend/src/api/campusMachineApi.ts 读取。接口地址是 <https://api.cleverschool.cn/washapi4/device/status>。请求使用 POST, body 包含 towerKey 和空的 deviceType。响应中的 macUnionCode 用来拆出机器编号, tower 和 floorName

用来生成位置，`status` 用来生成机器状态和剩余时间。状态文字中的“待机”“空闲”“可用”“可使用”会转成 `available`；“工作”“运转”“使用”“运行”会转成 `running`；“脱水”“开盖”“出错”“错误”“异常”“故障”“不可使用”“停用”会转成 `out_of_service`。剩余时间只从明确的“剩余时间”文字中读取。

海乐生活机器状态也由 `campusMachineApi.ts` 读取。接口地址是 <https://yshz-user.haier-ioc.com/position/deviceDetailPage>。请求使用 POST，body 包含 `positionId`、`categoryCode`、`page`、`floorCode` 和 `pageSize`。移动端会分别请求 00、01 和 02 三类设备，并把它们映射为普通洗衣机、洗鞋机和烘干机。响应中的 `id` 是机器编号，`name` 是位置文字，`state` 是状态。状态 1 表示可用，2 表示运行中，3 表示不可用；其他值记为 `unknown`。

Python 参考实现还保留楼栋和点位发现接口。CleverSchool 使用 <https://api.cleverschool.cn/washapi4/device/tower> 获取楼栋列表。海乐生活使用 <https://yshz-user.haier-ioc.com/position/nearPosition>，并用清华附近坐标 `lng=116.32697`，`lat=40.00281` 获取附近点位。后端会把同名楼栋合并成 `MachineTower`，并在 `provider_keys` 中保留不同来源的内部 `id`。

实时机器接口不提供完整价格和程序时，系统不会在机器解析层补齐这些业务数据。价格、模式和时长来自 `config/machine_rules.json` 以及移动端镜像的 `frontend/src/api/pricingRules.ts`。本地网页预览时，请求会改写到 `/cleverschool-api` 和 `/haier-api` 代理路径；Android APK 中使用 Capacitor HTTP 直接发起请求。

6.3 密钥与本地配置

移动端 release APK 不内置真实 API key。用户在 Profile 中填写 ModelHub `baseUrl`、`apikey` 和 `model_name`，配置保存在本机，并提供清除入口。

Python 参考实现使用本地 `config/api_config.json`，仓库只提交 `config/api_config.example.json`。真实 API key、`.env`、`keystore`、签名密码和真实用户数据都不应提交到仓库。

6.4 不静默猜测业务数据

项目规则强调 `no fallback / no default`。这里的含义是：缺少业务数据时，应显式展示未配置、不可用、未知、错误状态或抛出异常，而不是补一个看似合理但没有来源的数据。不能凭空生成材质、颜色、机器状态、天气、价格、时长、风险或洗衣方案。

7 运行方法与交付文件

7.1 本地运行与构建命令

项目仓库地址为 <https://github.com/vegetable123q/Wash-agent>。debug APK 交付路径为 `frontend/android/app/build/outputs/apk/debug/app-debug.apk`。如果本地尚未生成 APK，可按下面命令重新构建。

```
uv sync
uv run python -m unittest discover -v

cd frontend
```

```
npm install
npm test
npm run build
npm run cap:sync
```

构建 debug APK:

```
cd frontend
npm run apk:debug
```

生成路径:

```
frontend/android/app/build/outputs/apk/debug/app-debug.apk
```

7.2 自动检查与发布

`.github/workflows/preview.yml` 用于 preview 检查。它会同步 Python 环境, 运行 Python 单元测试, 使用 `npm ci` 安装前端依赖, 运行前端测试, 并执行前端 build。

`.github/workflows/release-apk.yml` 用于 main 分支发布。它会运行 Python 和前端测试, 从 `frontend/package.json` 读取 release version, 更新 Android version, 安装 JDK 21 和 Android SDK 35, 执行 Capacitor sync, 构建签名 release APK, 上传 artifact, 并发布 GitHub Release。

稳定 release 签名依赖 GitHub Secrets: `ANDROID_KEYSTORE_BASE64`、`ANDROID_KEYSTORE_PASSWORD`、`ANDROID_KEY_ALIAS`、`ANDROID_KEY_PASSWORD`。如果这些 secrets 缺失, CI 可以生成临时 key 来构建可安装 APK, 但该 key 不适合作为长期更新签名。

8 测试与验收结果

8.1 自动化测试与覆盖

当前仓库包含 Python unittest 和 frontend Vitest。后端测试覆盖衣物抽取、衣柜 CRUD、频率建议、校园机器解析、校园上下文、天气、洗衣规划、报告生成和完整集成流程。前端测试覆盖 API services、页面渲染、Profile 配置、ModelHub 配置持久化、脏衣篮、完成洗衣记录与撤销、推荐机器高亮、报告周统计、刷新行为和 App integration。

8.2 实测验收结果

`evaluation_system.md` 记录了 2026-05-28 的验收结果。主要结论如下:

- Python unittest: 通过, 命令为 `uv run python -m unittest discover -v`。
- Frontend test: 通过, 命令为 `npm test -- --run`。
- Frontend build: 通过, 命令为 `npm run build`。
- Capacitor sync: 通过, 命令为 `npm run cap:sync`。
- APK debug build: 通过, 命令为 `npm run apk:debug`, 生成 `frontend/android/app/build/outputs/apk/debug/app-debug.apk`。

- **APK 安装与启动:** 通过, Android Emulator 安装后可打开 `edu.tsinghua.washmatecampus/.MainActivity`。
- **APK 内置服务:** 通过, 不启动本机后端也可以查看 Today、衣柜、洗衣房、方案和报告等主流程。
- **WebView 冒烟测试:** 通过, 覆盖 Today、衣柜、衣物详情、添加衣物、洗衣房、机器详情、方案详情、报告和我的页面。

本轮验收发现的 P1 问题已修复并复验, 包括衣柜删除、宿舍楼选择、图片识别配置、展示型按钮禁用、空衣柜状态、固定样例详情、APK 后端依赖、API key 配置和等待时间提醒等。当前交付结论为: 自动化、模拟器 APK、API 和移动端主流程均通过, 未发现剩余 P0/P1 缺陷。

9 团队分工

9.1 成员与贡献预留

以下表格用于填写最终团队分工和贡献说明。成员名单已按提交信息预置, 具体负责模块和完成内容可在最终版中补充。

成员	负责模块	主要贡献
武子皓	待补充	待补充
王成思宇	待补充	待补充
林恺鎏	待补充	待补充
王博翔	待补充	待补充
周逸航	待补充	待补充

9.2 模块维护说明

- 页面和交互放在 `frontend/src/screens/`, 通用外壳或组件放在 `frontend/src/components/`。
- 移动端业务逻辑放在对应 `frontend/src/api/ service` 中。
- Python 参考逻辑放在对应 `backend package` 中, 例如 `clothing extraction`、`wardrobe`、`campus`、`laundry` 或 `reports`。
- Python 共享模型只放在 `backend/shared/models.py`。
- 移动端共享类型放在 `frontend/src/api/types.ts`。
- 如果修改模块边界、公共接口、配置行为或主流程, 应同步更新 `docs/structure_design.md`。

9.3 开发注意事项

- 不新增旧路径兼容层，例如重新创建 flat backend import。
- 不把提示词、解析器、洗衣规则、存储细节或报告拼装写进页面。
- 不为普通衣柜、脏衣篮、方案和报告流程引入隐藏的远程后端依赖。
- 不提交 API key、keystore、签名密码或真实用户数据。
- 不通过猜测结果掩盖外部 API 失败或缺失配置。

10 当前限制与未来改进

当前实现已经适合作为完整移动端产品演示，但仍有明确扩展空间：

- 账号体系和云端同步尚未实现；当前衣柜、profile、脏衣篮和完成洗衣记录以本机存储为主。
- 如果未来要支持多设备同步，可以增加 hosted backend，但不应破坏当前 APK 内置业务服务的清晰边界。
- 如果要支持更多 ModelHub 模型，应通过显式 allow-list、UI 控件和测试来扩展。
- 如果要支持更多校园机器 provider，应放在 campus API/context 边界内，并继续向 UI 暴露用户可读楼名而不是 provider id。
- 如果要导出更多报告格式，应消费结构化 WashReport，不要解析自然语言段落。

10.1 总结

WashMate Campus 的事实实现是一个以移动端为主的完整产品。前端不仅是 UI，也包含 APK 内置业务服务；Python 后端是参考实现和测试基准，帮助团队保持清晰的模块边界、数据结构和业务规则。这样的设计让项目能直接演示完整产品流程，也能通过测试和文档保留可维护的工程结构。